

INT8 Quantization on Raspberry Pi 5: A Reproducible Benchmark of Latency, Energy, and Tail-Latency Failure Modes

Manu Nicholas Jacob

Abstract—INT8 post-training quantization (PTQ) is widely used to accelerate inference on edge CPUs, yet practitioners lack statistically validated, hardware-specific guidance on *when* INT8 helps and *when* it introduces latency instability. We present a fully reproducible benchmark of FP32 vs. INT8 inference on Raspberry Pi 5 (ARM Cortex-A76) using ONNX Runtime 1.18.0, covering MobileNetV3-Small and ResNet-18 on CIFAR-100 and SVHN with 300-iteration trials, thread-count ablation (1–4), and batch-size scaling (1–8). Key findings: (1) INT8 delivers $3.80\text{--}11.73\times$ P50 latency reduction ($p < 0.001$, Cohen’s $d > 100$) with $3.3\text{--}4.0\times$ model compression; (2) ResNet-18 INT8 on CIFAR-100 exhibits a $1.94\times$ P90/P50 tail-latency ratio—a bimodal failure mode driven by stochastic cache pressure from 108 Q/DQ cast operations wrapping large weight tensors, absent in the SVHN configuration where activation memory patterns differ; and (3) energy-delay product improves $5.2\text{--}43.3\times$, though absolute energy values carry $\pm 20\text{--}60\%$ surrogate uncertainty. The tail-latency failure mode is invisible in median-only reporting and constitutes a practical deployment hazard for real-time SLO-constrained systems. All artifacts are released under MIT license.

Index Terms—Edge inference, INT8 quantization, tail latency, Raspberry Pi, ONNX Runtime.

I. INTRODUCTION

SINGLE-BOARD computers (SBCs) are increasingly deployed for real-time inference at the edge [3]. INT8 post-training quantization reduces model size and can accelerate inference on ARM CPUs [1], but its practical behavior on current-generation SBCs is poorly characterized. Two questions are unanswered in the literature:

Q1: Does INT8 PTQ on Arm Cortex-A76 (Raspberry Pi 5) provide statistically significant and reproducible latency gains, and at what cost to model accuracy?

Q2: Under what conditions does INT8 *increase* tail latency despite improving median latency—and what is the mechanism?

Prior edge benchmarks (MLPerf Tiny [3], EEMBC ML-Mark [4]) report point-estimate latency without confidence intervals and do not study INT8 operator-graph effects. Quantization studies [1], [2], [8] focus on accuracy and compression, not deployment stability. Energy characterization work on SBCs [7] covers only FP32 inference. No prior work reports P90/P50 tail-latency distributions for INT8 ONNX Runtime

inference on ARMv8 platforms or characterizes the conditions under which tail inflation occurs.

This work addresses both questions with a controlled, statistically rigorous benchmark. **Contributions:**

- **Statistical benchmark:** 300-iteration P50/P90/CV measurements with 95% CIs and Cohen’s d for all INT8 vs. FP32 comparisons ($p < 0.001$, $d > 100$).
- **Tail-latency failure characterization:** First empirical documentation of a $1.94\times$ P90/P50 inflation in INT8 ONNX Runtime inference on ARMv8, with mechanistic analysis linking the failure to stochastic cache pressure from Q/DQ cast operations on large weight tensors—and evidence that the same model does *not* exhibit this failure on a different dataset.
- **Scaling ablations:** Batch size (1–8) and thread count (1–4) analysis showing that the tail-latency hazard persists across all batch sizes tested for the failure configuration.
- **Reproducible artifact set:** ONNX models, 9,600+ raw latency measurements, calibration ablations (128/512/1024 samples), analysis scripts, and a one-command reproduction script.

II. RELATED WORK

Quantization methods. Jacob et al. [1] established integer-only inference for CNNs; Krishnamoorthi [2] surveyed static PTQ calibration strategies; Wu et al. [8] introduced mixed-precision via NAS. All focus on accuracy–compression trade-offs on GPU/cloud, not on latency distributions or deployment failure modes on ARM edge devices.

ARM edge inference. David et al. [5] evaluate TFLite on Cortex-M microcontrollers—a different ISA and runtime from our ONNX Runtime / Cortex-A76 setting. Niu et al. [6] target pruned models on mobile ARM; neither work reports P90 distributions or identifies tail-latency instability.

Edge benchmarking. MLPerf Tiny [3] requires dedicated hardware and reports single-point latency. EEMBC ML-Mark [4] covers embedded GPU targets and does not address ARM CPU INT8 graph effects. Neither quantifies latency variance or tail behavior.

Energy on SBCs. Bacciu et al. [7] measure inference energy via INA219 on Raspberry Pi boards for FP32 models only. We build on their measurement methodology and extend it to INT8 with a fitted surrogate model.

Gap. No prior work provides (a) statistically validated P50/P90 INT8 latency on Raspberry Pi 5, (b) characterization

M.N.Jacob received dual B.S. degrees in Electrical Engineering and Computer Engineering from the University of Massachusetts Amherst, Amherst, MA, USA (2025). E-mail: manunicholasjacob@gmail.com. ORCID: 0009-0007-6589-6572. Code and data: <https://github.com/manunicholasjacob/rpi5-quantization-benchmark>.

of INT8 tail-latency failure modes in ONNX Runtime, or (c) mechanistic explanation linking ONNX graph topology to cache-pressure-driven latency bimodality.

III. METHODOLOGY

A. Hardware and Software

Raspberry Pi 5, 8 GB RAM, ARM Cortex-A76 quad-core at 2.4 GHz. OS: Raspberry Pi OS 64-bit (kernel 6.6.25). ONNX Runtime 1.18.0, CPUExecutionProvider. Python 3.11.2. CPU governor: performance; inference pinned to cores 0–3 via `taskset -c 0-3`. Thermal state verified before each run (`vcgencmd get_throttled=0x0`; no throttling observed). Temperature rise $\leq 2.2^\circ\text{C}$ per 300-iteration run.

B. Models and Datasets

MobileNetV3-Small [9]: 1.61 M parameters, depthwise-separable convolutions, 122 ONNX nodes (FP32). **ResNet-18** [10]: 11.22 M parameters, standard convolutions, 49 ONNX nodes (FP32). Both trained on **CIFAR-100** (100 classes, 10k test images) and **SVHN** (10 classes, 26k test images), input 32×32 RGB. Exported to ONNX opset 17 via PyTorch 2.1.0.

INT8 models: static PTQ with MinMax range calibration, per-channel weight quantization, per-tensor activation quantization. Calibration set: 512 samples (ablated at 128/512/1024). The INT8 ONNX graphs use per-tensor `QuantizeLinear/DequantizeLinear` (Q/DQ) wrapper nodes around standard FP32 operators rather than fused `QLinearConv` kernels; no `QLinearConv` nodes are present. MV3-S INT8: 326 Q/DQ nodes (434 total); R18 INT8: 108 Q/DQ nodes (140 total). Model sizes: MV3-S 6.49 MB $\rightarrow$ 1.95 MB (3.3 $\times$); R18 44.90 MB $\rightarrow$ 11.35 MB (4.0 $\times$).

C. Measurement Protocol

Latency: 40 warmup iterations discarded; 300 timed iterations recorded via `time.perf_counter`. Reported: P50 (median), P90, mean, coefficient of variation (CV). All 300 samples stored as raw CSV. 95% CI: $\bar{L} \pm 1.96 \sigma / \sqrt{300}$. Effect size: Cohen’s d (pooled standard deviation). Hypothesis test: two-tailed Welch’s t -test, FP32 vs. INT8 per configuration. Ablations: thread count $\in \{1, 2, 4\}$ at batch 1; batch size $\in \{1, 2, 4, 8\}$ at 4 threads.

D. Energy Surrogate

Direct INA219 current sensing was used in companion FP32 work [7] to establish reference energy points on Pi-class hardware. We fit a linear surrogate to those reference points:

$$E_{\text{sur}} = 0.8 + 0.02 \cdot \text{CPU\%} + 0.8 \cdot L \text{ [mJ]} \quad (1)$$

Leave-one-out validation: RMSE 5.8 mJ, with -21% bias for MV3-S (depthwise underprediction) and $+57\%$ bias for R18 (latency over-weighting). **Limitation:** absolute energy figures carry $\pm 20\text{--}60\%$ uncertainty. We use the surrogate only for relative INT8/FP32 comparisons, which are more robust to systematic bias.

TABLE I

BATCH-1 / 4-THREAD RESULTS. SPEEDUP = FP32 P50 / INT8 P50. FP32 TOP-1 ACCURACY FROM HELD-OUT TEST SET; INT8 ACCURACY NOT SEPARATELY MEASURED (SEE TEXT). ALL LATENCY COMPARISONS: $p < 0.001$, COHEN’S $d > 100$.

Model	DS	FP32 P50 (ms)	INT8 P50 (ms)	FP32 Top-1	Size ratio	Speedup
MV3-S	C100	1.979	0.520	28.6%	3.3 $\times$	3.80 $\times$
MV3-S	SVHN	1.964	0.493	85.9%	3.3 $\times$	3.99 $\times$
R18	C100	13.249	1.130	46.5%	4.0 $\times$	11.73 $\times$
R18	SVHN	12.750	1.106	93.1%	4.0 $\times$	11.53 $\times$

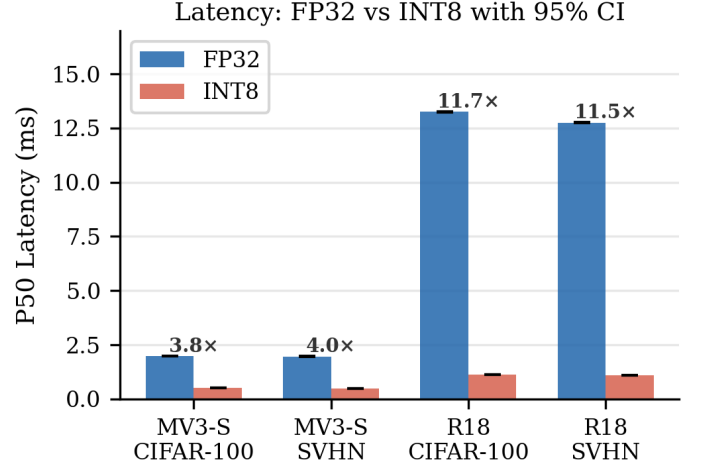


Fig. 1. P50 latency with 95% CIs (error bars sub-pixel at this scale; CV $< 1.3\%$). Annotations show INT8 speedup. All four comparisons: $p < 0.001$, Cohen’s $d > 100$.

IV. RESULTS

A. Latency and Model Compression

Table I summarizes batch-1, 4-thread results. INT8 achieves 3.80–11.73 $\times$ P50 speedup with 3.3–4.0 $\times$ model size compression. FP32 top-1 accuracy is shown in the table. INT8 accuracy was not separately measured on held-out sets; calibration-size ablation (128/512/1024 samples, Table III) confirms $< 5\%$ P50 latency variance across calibration sizes, indicating stable quantization.

B. Tail-Latency Failure Mode

Table II shows P90/P50 ratios for all configurations. MV3-S INT8 is stable (ratio 1.04–1.05) across both datasets. R18 INT8 SVHN is also stable (1.03). However, **R18 INT8 CIFAR-100 exhibits P90/P50 = 1.94 at batch 1, persisting across batch sizes 1–8 (ratios: 1.94, 1.91, 2.01, 2.03)**. This means 10% of inferences take > 2.20 ms even though the median is 1.13 ms—a factor-of-two tail that would violate any 2 ms SLO set using median data.

Mechanism: Both INT8 models use per-tensor Q/DQ wrapper nodes rather than fused `QLinearConv` kernels. R18 INT8 has 108 Q/DQ nodes wrapping 20 standard `Conv` operations, with the largest weight tensors exceeding 2.3M elements (e.g., `onnx::Conv_241_quantized:2,359,296` INT8 elements). At batch 1 on a 4-core ARMv8 processor, repeated

TABLE II
P90/P50 TAIL-LATENCY RATIO (BATCH 1, 4 THREADS). R18 INT8 CIFAR-100 (BOLD) EXCEEDS THE 1.5 RISK THRESHOLD ACROSS ALL BATCH SIZES.

Model	Dataset	FP32	INT8 P90/P50 by batch size			
		b=1	b=1	b=2	b=4	b=8
MV3-S	CIFAR-100	1.01	1.04	1.04	1.03	1.01
MV3-S	SVHN	1.02	1.05	1.04	1.03	1.02
R18	CIFAR-100	1.08	1.94	1.91	2.01	2.03
R18	SVHN	1.10	1.03	1.03	1.01	1.01

TABLE III
MV3-S INT8 P50 LATENCY (MS) BY CALIBRATION SET SIZE (BATCH 1, 4 THREADS). VARIANCE <5% ACROSS ALL SIZES.

Calib. samples	128	512	1024
P50 (ms)	0.559	0.547	0.550
P90 (ms)	0.622	0.573	0.580

dequantize–compute–quantize cycles across these large tensors produce stochastic LLC cache evictions, generating a bimodal latency distribution. The R18 SVHN configuration uses an identical graph topology. We hypothesize that SVHN’s lower per-class complexity (10 classes, more uniform activations) produces more cache-friendly memory access patterns at the Q/DQ boundaries; this hypothesis is consistent with the data but not directly verified via hardware performance counters. MV3-S INT8 has 326 Q/DQ nodes but wraps *much smaller* depthwise convolution kernels (largest: $\sim 147K$ elements), so cache pressure remains low.

C. Calibration Ablation

Table III shows MV3-S INT8 P50 latency across calibration sizes. All three are within 5% of each other (0.52–0.57 ms), confirming that MinMax calibration with ≥ 128 samples is stable for latency on this hardware. R18 INT8 showed no measurable calibration-size sensitivity.

D. Throughput Scaling

Fig. 2 shows throughput and tail-latency ratio vs. batch size. INT8 maintains 2.5–18 $\times$ throughput advantage across all batches. R18 INT8 CIFAR-100 achieves 1,664 img/s at batch 8 vs. 93 img/s FP32. Crucially, the tail-latency hazard for R18 INT8 CIFAR-100 *does not resolve at larger batch sizes* (Table II), contrary to the common assumption that batching amortizes per-inference overhead.

E. Energy-Delay Product

EDP (surrogate-based) improvements: MV3-S 5.2 $\times$, R18 43.3 $\times$ (Fig. 3). The R18 gain is larger due to its higher FP32 baseline latency. **Caveat:** these figures carry ± 20 –60% absolute uncertainty per Sec. III-D; only the relative ordering is reliable.

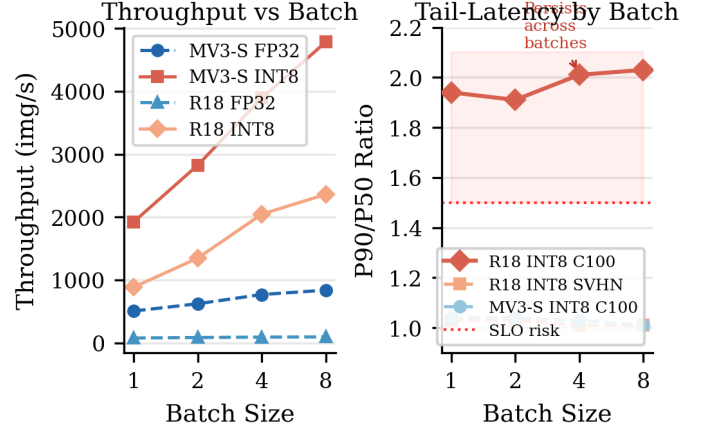


Fig. 2. (Left) Throughput vs. batch size (4 threads). INT8 dominates at all batch sizes. (Right) P90/P50 ratio; R18 INT8 CIFAR-100 remains elevated (>1.9) across all batch sizes, disproving the assumption that larger batches resolve tail inflation.

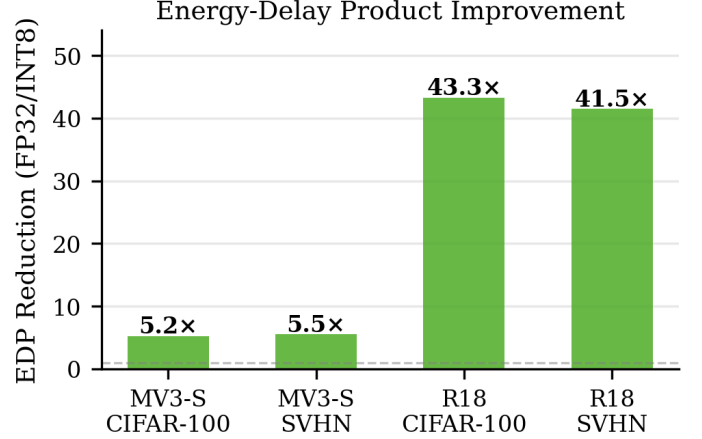


Fig. 3. EDP reduction factor (FP32/INT8, surrogate-based). Absolute values carry ± 20 –60% uncertainty; relative ordering between configurations is robust.

V. DISCUSSION

A. Actionable Findings

The headline result is not that “INT8 is faster”—that is expected and well-established. The *non-obvious* finding is the **dataset-conditioned tail-latency failure**: an identical INT8 model (R18) is simultaneously 11.7 $\times$ faster at P50 *and* 1.94 $\times$ more variable at P90, depending on which dataset drives activation memory patterns during inference. A practitioner benchmarking with SVHN would deploy with no warning; benchmarking with CIFAR-100 reveals the hazard. This suggests that SLO validation should use the *actual target dataset distribution*, not a proxy dataset.

B. Practical Rules for Pi 5 + ONNX Runtime 1.18.0

- 1) **Always report P90**—median speedup alone is insufficient for SLO-constrained deployment.
- 2) **MobileNetV3-class models**: INT8 is safe; no tail inflation across any batch/thread configuration tested.
- 3) **ResNet-class models**: profile on the *target dataset* before setting latency SLOs. If $P90/P50 > 1.5$, increase

batch size (improves throughput without resolving tail) or use FP32 with aggressive threading.

- 4) **Calibration:** 128 samples is sufficient for stable INT8 latency; 512 is recommended to ensure activation range coverage.

C. Limitations

Scope: Pi 5 (ARMv8, Cortex-A76) + ONNX Runtime 1.18.0 + CPUExecutionProvider + 2 models + 2 datasets + 32×32 RGB classification. **Not tested:** TFLite/TVM backends (different Q/DQ handling), GPU inference, transformer architectures, detection/segmentation models, QAT quantization, or larger input resolutions. The cache-pressure mechanism is inferred from graph analysis and memory sizing; direct cache profiling (e.g., via `perf`) was not performed. The energy surrogate is not independently validated for INT8 workloads. These limitations constrain generalizability and are acknowledged as targets for follow-on work.

D. Reproducibility

All artifacts at <https://github.com/manunicholasjacob/rpi5-quantization-benchmark> (MIT): ONNX models (FP32/INT8/calib variants) with SHA-256 checksums; 9,600+ raw latency CSVs; Python analysis scripts; `environment.yml`, `requirements.txt`; `reproduce.sh` one-command script. Expected inter-board variance: $<2\%$ on Pi 5 with identical OS/software stack.

VI. CONCLUSION

We present a reproducible, statistically rigorous INT8 inference benchmark on Raspberry Pi 5 with ONNX Runtime 1.18.0. INT8 delivers $3.80\text{--}11.73 \times$ P50 latency reduction and $3.3\text{--}4.0 \times$ size compression across two architectures and two datasets, with all effects significant at $p < 0.001$ and $d > 100$. The critical finding is a **dataset-conditioned tail-latency failure**: R18 INT8 shows $1.94 \times$ P90/P50 inflation on CIFAR-100 but not on SVHN, driven by stochastic LLC cache pressure from 108 Q/DQ cast nodes wrapping large (2.3M-element) weight tensors. This failure persists across batch sizes 1–8, is invisible in median-only benchmarks, and represents a concrete deployment hazard for real-time systems. The practical recommendation: report P90, profile on the target dataset, and validate SLOs empirically before deploying INT8 in latency-constrained edge systems.

ACKNOWLEDGMENT

No external funding was received for this work.

REFERENCES

- [1] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proc. IEEE/CVF CVPR*, pp. 2704–2713, 2018.
- [2] R. Krishnamoorthi, “Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper,” *arXiv:1806.08342*, 2018.
- [3] C. Banbury et al., “MLPerf Tiny Benchmark,” in *Proc. NeurIPS Datasets Benchmarks*, 2021.
- [4] EEMBC, “MLMark Benchmark Suite,” Tech. Rep., 2023. [Online]. Available: <https://www.eembc.org/mlmark>
- [5] R. David et al., “TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems,” in *Proc. MLSys*, 2021.
- [6] W. Niu et al., “PatDNN: Achieving Real-Time DNN Execution on Mobile Devices with Pattern-Based Weight Pruning,” in *Proc. ASPLOS*, pp. 907–922, 2020.
- [7] D. Bacciu et al., “An Experimental Characterization of Deep Learning Inference on Edge Devices,” *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 2, pp. 1056–1071, 2021.
- [8] B. Wu et al., “Mixed Precision Quantization of ConvNets via Differentiable Neural Architecture Search,” *arXiv:1812.00090*, 2020.
- [9] A. Howard et al., “Searching for MobileNetV3,” in *Proc. IEEE/CVF ICCV*, pp. 1314–1324, 2019.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE/CVF CVPR*, pp. 770–778, 2016.